



INDIAN INSTITUTE OF INFORMATION TECHNOLOGY,
DESIGN AND MANUFACTURING,
KANCHEEPURAM



DIGITAL BRAINS

VERILOG HDL

SYSTEM CODING CLUB





SYSTEM CODING
CLUB
— TOGETHER WE LEARN —



Motivation

- This session introduces you to **Verilog HDL**, the language behind modern digital hardware like CPUs, memory, and embedded systems.
- Understanding Verilog gives you a **practical foundation** in how real circuits are designed, simulated, and implemented.
- It connects your academic concepts in digital logic with **industry-grade workflows** used in chip design.
- Gaining Verilog skills now gives you a **competitive edge** for core roles in VLSI, embedded systems, and semiconductor industries.
- Most importantly, this session helps you **think like a hardware engineer**—translating logic into actual working hardware



SYSTEM CODING
CLUB
— TOGETHER WE LEARN —



Prerequisites

Basic knowledge of Digital Circuits.



SYSTEM CODING
CLUB
— TOGETHER WE LEARN —



What is HDL & Verilog HDL?

A **Hardware Description Language (HDL)** allows you to describe digital circuits in code form.

Verilog HDL is a specific HDL used to **design, simulate, and implement** digital systems like CPUs, controllers, and logic blocks.

- Developed in **1984** by **Phil Moorby**
- Standardized by **IEEE** in **1995 (IEEE 1364)**
- Verilog lets you write **hardware as code**



BASIC EXAMPLE

```
module nand_gate(input A, B, output Y);  
    assign Y = ~(A & B);  
endmodule
```

- **module** defines a block called **nand_gate**
- **input** and **output** declare signal directions
- **assign** uses $\sim(A \& B)$ to model NAND logic

This Is The Simplest Logic Gate And A Great Starting Point In Verilog.



TYPES OF MODELLING: STRUCTURAL AND

BEH

Aspect	Structural Modeling	Behavioral Modeling
Describes	Circuit connection (gate-level)	Circuit behavior (logic-level)
Code Style	Instantiates basic gates	Uses <code>always</code> blocks & expressions
Abstraction	Low-level	High-level (RTL design)
Use Case	Gate simulation, layout	Algorithmic logic, quick testing

Use **behavioral** for design, **structural** for logic verification.



D LATCH- STRUCTURAL vs BEHAVIORAL

BEHAVIORAL:

```
module d_latch_behav(input D, EN, output reg Q);  
    always @ (D or EN)  
        if (EN) Q = D;  
endmodule
```

STRUCTURAL:

```
module d_latch_structural(input D, EN, output Q);  
    wire S, R, Qa, Qb;  
  
    nand(S, D, EN);  
    nand(R, ~D, EN);  
    nand(Qa, S, Qb);  
    nand(Qb, R, Qa);  
  
    assign Q = Qa;  
endmodule
```

Behavioral is shorter, easier to simulate. Structural mimics actual gates.

Structural type codes are easier to debug, unlike behavioral type codes.



SYSTEM CODING
CLUB
— TOGETHER WE LEARN —



QUIZ

Write the Verilog assign statement for a NAND gate with inputs P, Q and output R.



**SYSTEM CODING
CLUB**
— TOGETHER WE LEARN —



VERILOG TO ASIC – STEP-BY-STEP (PART 1)

After Verilog code is written, it is converted into real hardware via a multi-step process.

1. RTL Coding: You write modules (e.g., full adder)
 2. Simulation: Use ModelSim or Icarus to verify logic
 3. Synthesis: RTL is converted into gate-level netlist
 4. Tools: Yosys (open), Synopsys DC (industry)
- ✓ After synthesis, hardware structure begins to take form.



SYSTEM CODING
CLUB
— TOGETHER WE LEARN —



5. Floorplanning & Placement: Physical arrangement of blocks

6. Routing: Connections between placed cells are wired

7. DRC/LVS: Design Rule and Layout checks done

8. GDSII Generation: Layout sent to silicon foundry for fabrication



This entire flow starts from your **Verilog file**



BASIC VERILOG FEATURES – CONDITIONAL, CASE, LOOPS

Conditional Statement (if-else):

if (EN) Y = D; else Y = 0;

Case Statement:

case(sel) 2'b00: Y = A; 2'b01: Y = B; endcase

Loop (for):

for (i=0; i<8; i=i+1) mem[i] = 0;

These are useful in writing **clean, compact behavioral code**.



SYSTEM CODING
CLUB
— TOGETHER WE LEARN —



Example 1 – Full Adder (Structural)

```
assign SUM = A ^ B ^ Cin;  
assign Cout = (A & B) | (B & Cin) | (A & Cin);
```

Each line represents a logic operation

XOR gives the **sum**, AND+OR gives **carry**

✓ This mimics the **actual full adder logic circuit**



Example 2 – 4-bit Adder (Behavioral)

```
module adder_4bit(input [3:0] A, B, output [4:0] SUM);  
  assign SUM = A + B;  
endmodule
```

Adds two 4-bit numbers, stores 5-bit result

Verilog handles **bit-width** and **overflow** automatically

✓ Cleanest way to model arithmetic logic



Example 3 – 8x1 Multiplexer (Structural)

assign Y = ($\sim S2$ & $\sim S1$ & $\sim S0$ & I0) | ... | (S2 & S1 & S0 & I7);

Uses full expression of select lines

Each combination selects a unique input line

✓ Structural MUX teaches gate-level **OR-AND** decoding



Example 4 – 3x8 Decoder (Behavioral with Loop)

```
for (i = 0; i < 8; i = i + 1) Y[i] = (i == sel) ? 1 : 0;
```

Loop runs through all 8 outputs

Sets only the matched index to 1

✓ Elegant way to build decoders without long case/if chains



Example 6 – D Flip-Flop (Behavioral)

always @ (posedge CLK) Q <= D;

Triggered on **rising edge** of clock

Captures and stores D input into Q output

✓ This is the **basic storage element** in all digital systems



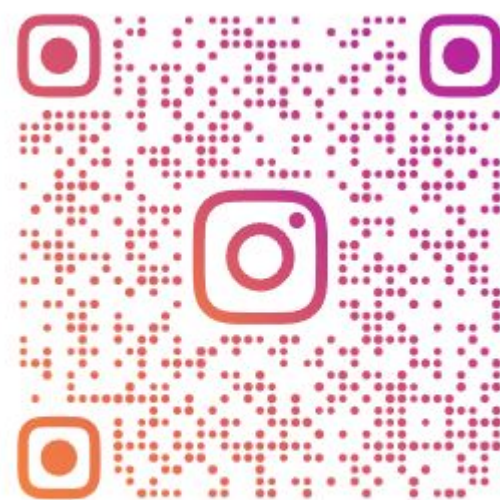
**SYSTEM CODING
CLUB**
— TOGETHER WE LEARN —



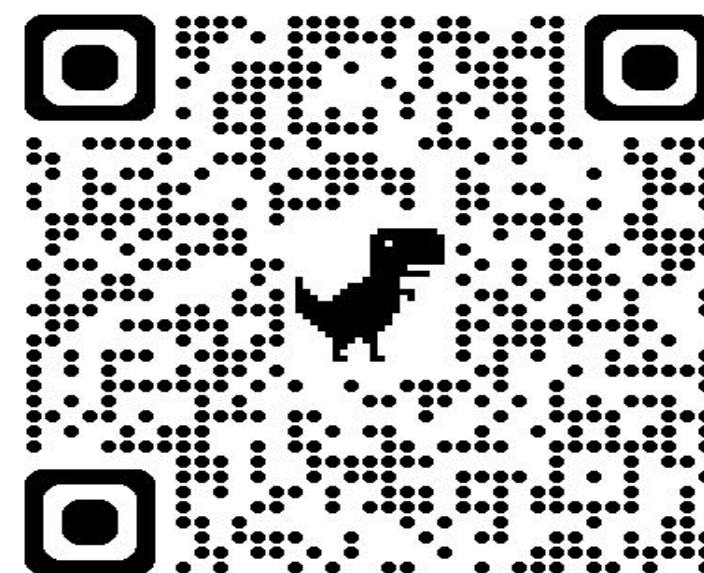
Contact us



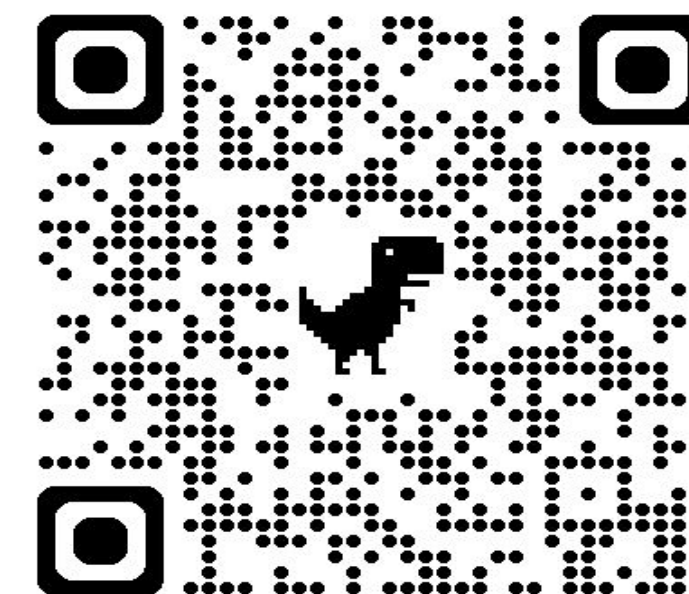
SYSTEMCODINGCLUB



IIITDM.TECHNICAL



**SCC LINKEDIN
LINKEDIN**



TECH AFFAIRS



Thank You

